Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Université de Carthage

Institut Supérieur des Technologies de l'Information et de la Communication

Classe : GLSI-3	**Examen TP Virtualisation & Cloud**	**Nombre de pages : 3**
Enseignantes : dhekra El hamdi	**Date : 12 Décembre 2025**	**Durée : 1h**

OBJECTIF :

Déployer une application à 3 services :

- **Frontend** : serveur web (Nginx)
- **Backend** : API Node.js (Express)
- **Base de données** : MariaDB

L'examen TP est basé sur les images légères node-alpine.tar, nginx:alpine, mariadb:10.6.

PARTIE 1 – STRUCTURE DU PROJET

Fichier fourni :

- un fichier ZIP nommé **TP_Docker_Exam_Files.zip**.
- Trois fichiers Tar pour les **images de base** : node-alpine, nginx:alpine, mariadb:10.6.

Vous trouverez ce fichier dans **Classeroom – Virtualisation & Cloud ou via clé usb**.

Le fichier **TP_Docker_Exam_Files.zip** contient exactement les fichiers :

TP_Docker_Exam_Files_modified.zip

```
├─ frontend/
    └─ index.html
├─ backend/
    ├─ app.js
    ├─ package.json
```

├─ db/

├─ schema.sql

PARTIE 2 – BACKEND (Dockerfile à construire)

Votre conteneur backend doit :

- Utiliser node:18-alpine
- Installer les dépendances depuis package.json : RUN npm install --production
- Copier le code app.js
- Exposer le port **3000**
- Lancer l'app avec : node app.js
- Nom de l'image : **backend**

Travail demandé : rédiger backend/Dockerfile et builder ensuite votre image nommée backend.

NB : L'image Docker node-alpine nécessaire au build du backend est déjà fournie dans un fichier TAR. Vous pouvez la charger directement depuis le fichier disponible sans avoir besoin d'une connexion Internet en utilisant la commande : docker load -i node-alpine.tar

PARTIE 3– FRONTEND

frontend/Dockerfile. :

```
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/
```

NB : L'image Docker nginx:alpine nécessaire au build du frontend est déjà fournie dans un fichier TAR.: docker load -i nginx:alpine.tar

PARTIE 4 – QUESTIONS DE MANIPULATION DOCKER

- Afficher toutes les images Docker.
- Supprimer une image Docker inutilisée dans cet examen.
- Afficher tous les conteneurs.
- Tagger l'image frontend pour Docker Hub.
- Se connecter à Docker Hub en ligne de commande.
- Publier l'image taggée sur Docker Hub.

PARTIE 5 – BASE DE DONNÉES

db/ Dockerfile :

```
FROM mariadb:10.6
```

NB : L'image Docker mariadb:10.6 nécessaire au build du frontend est déjà fournie dans un fichier TAR: docker load -i mariadb:10.6.tar

PARTIE 6– DOCKER COMPOSE (à construire)

Vous devez construire **le fichier docker-compose.yml complet**

Le fichier doit :

- déclarer les **3 services** (frontend, backend, db)
- Le service de base de données doit utiliser un volume nommé db-data pour stocker les données de MariaDB.
  Vous pouvez déclarer ce volume à la fin du fichier docker-compose.yml :

  volumes:

  db-data:

  Puis l'utiliser dans la section db :

  volumes:

  - db-data:/var/lib/mysql

  - Pour que MariaDB exécute automatiquement votre fichier SQL (schema.sql) lors du premier démarrage du conteneur,

  - ./schema.sql:/docker-entrypoint-initdb.d/schema.sql

- gérer les dépendances (depends_on)
- exposer les ports :
  - frontend → 8080:80
  - backend → 3000:3000
  - db → 3306:3306
- utiliser les images **buildées depuis vos Dockerfiles**

Important : Le service de base de données doit obligatoirement contenir les variables d'environnement suivantes dans le docker-compose.yml :

- MYSQL_ROOT_PASSWORD : root
- MYSQL_DATABASE : appdb
- MYSQL_USER : user
- MYSQL_PASSWORD : test

PARTIE 7 – LANCEMENT

Lancer l'ensemble des services définis dans votre fichier docker-compose.yml

PARTIE 8 – TESTS

Backend : http://localhost:3000/api/data

Frontend : http://localhost:8080

Bon travail ☺